Software Requirements Specification

for

Event Registration System

Prepared by Doaa Ayman, Mariam Saeed, Sarah Mohamed, Toka Abdelaziz, Salma Kamal

Helwan University

2026/10/05

Agenda

Table of Contents

PART 1: Overview & Software Requirements Specification

1. Introduction

1.1 Purpose

Event registration systems are specialized software platforms developed to support the planning, publication, registration, attendance tracking, and coordination activities of institutions, clubs, training centers, and event teams. This project seeks delivery of a centralized, user-friendly platform to automate these processes while ensuring secure access, accurate seat management, transparent event updates, and operational consistency.

Educational institutions and community organizations frequently depend on spreadsheets, manual lists, messaging groups, or disconnected web forms to manage events. Such methods often create duplicate registrations, late schedule communication, weak role control, poor visibility into participant counts, and unnecessary administrative effort. The Event Registration System bridges the gap between event administrators, organizers, and attendees by providing one platform that simplifies event lifecycle management.

An Event Registration System enables organizations to:

- **Simplify Event Discovery:** users can browse upcoming events, inspect schedules, view locations, and understand seat availability before deciding to participate.
- **Improve Registration Accuracy:** the system records each participant-event relationship in a structured form and prevents duplicate active registrations.
- **Support Organizer Coordination:** assigned organizers can focus on their own events and view participant counts without needing full system administration privileges.

- **Maintain Reliable Notifications:** event changes such as cancellation, rescheduling, organizer assignment, registration creation, and cancellation are communicated through stored notifications.
- **Reduce Manual Work:** admins can manage events, managed accounts, and organizer assignments without switching between unrelated tools.
- **Provide Clear Participation Records:** attendees can access their confirmed registrations in a ticket- style page and track their own participation history.

1.2 Project Scope

Our project aims to develop an Event Registration System tailored for academic, training, workshop, seminar, and club-based events. The system provides modules for authentication, event management, registration processing, organizer coordination, notifications, and role-specific user interaction. It is a web-based application accessible through standard desktop browsers and designed around Admin, Organizer, and Attendee roles.

Objectives and Deliverables

- Develop a web-based application for managing the creation, discovery, and participation lifecycle of events.
- Implement multi-role user access for Admin, Organizer, and Attendee users.
- Provide secure registration and login with token-based authentication and role-based authorization.
- Enable event creation, update, cancellation, rescheduling, and organizer assignment through controlled interfaces.
- Allow attendees to register for events, cancel participation, and view

confirmed tickets.

- ❖ Generate and store notifications for important event and registration changes.
- ❖ Provide administrative functionality for managed user creation and event supervision.
- ❖ Maintain a scalable project structure using a microservices architecture and container-friendly deployment layout.

Inclusions

- Secure login and registration system for supported roles.
- Role-specific dashboards and protected routes.
- Event creation, editing, cancellation, and rescheduling.
- Event browsing, detail display, and seat availability calculation.
- Event registration, cancellation, and ticket management.
- System-based notifications and read-state tracking.
- Admin dashboard for managing users and events.

Exclusions

- Online payment gateway integration for paid tickets.
- Native mobile application version of the platform.
- QR-based physical check-in or badge printing workflows.
- Multi-language support beyond English.
- External email or SMS notification delivery as the primary channel implemented.

Dependencies

- Availability of backend services for authentication, events, registrations, and notifications.
- Reliable relational database availability for all persistent modules.
- Correct gateway, discovery, and configuration setup.
- Access to project stakeholders and course expectations for requirements validation.
- Modern browser compatibility for frontend access.

1.3 Glossary and Abbreviations

Term/Abbreviation	Definition
ERS	Event Registration System
SRS	Software Requirements Specification
JWT	JSON Web Token used for stateless authentication
Admin	A privileged user responsible for supervising users, events, and assignments
Organizer	A user assigned responsibility for one or more events
Attendee	A participant who browses events and registers for attendance
API Gateway	The single external entry point for backend services
Config Server	The service that centralizes configuration values
Eureka	The service discovery server used by the microservices
Registration Status	The state of a registration such as REGISTERED or CANCELLED
Event Status	The state of an event such as SCHEDULED, RESCHEDULED, or CANCELLED
Notification	A stored user-facing system message
Ticket	A digital confirmed-registration representation shown to the attendee
RBAC	Role-Based Access Control
Managed User	An account created by an Admin on behalf of another user

Availability	The number of seats still open for an event
Fit Criteria	Measurable conditions used to verify a requirement
MoSCoW Scheme	Prioritization method using Must, Should, Could, and Won't

1.4 List of the System Stakeholders.

- **Administrators**: manage the platform, create managed users, assign organizers, and supervise event operations.
- **Organizers**: review assigned events, check participant lists, and stay aware of schedule or assignment changes.
- **Participants (Attendees):** register and log into the system, browse available events, register for events, cancel registrations, and receive notifications about updates and changes.
- **Developers:** build, maintain, test, document, and evolve the system.
- Course Instructors: evaluate the project's engineering quality, documentation, and implementation alignment.
- **Institutional or Club Coordinators:** represent real-world event management needs and workflows.
- **Database and Infrastructure Operators:** maintain service execution, routing, persistence, and containerized deployment.

1.5 References

- Ian Sommerville, (2020). Engineering Software Products – An Introduction to Modern Software Engineering. Pearson.

- Learn Microservices with Spring Boot 3
  Macero García, M., & Telang, T. (2023). Learn microservices with Spring Boot 3: A practical approach using event-driven architecture, cloud-native patterns, and containerization (3rd ed.). Apress. https://doi.org/10.1007/978-1-4842-9757-5

- Practical Microservices Architectural Patterns
  Christudas, B. A. (2019). *Practical microservices architectural patterns: Event-based Java microservices with Spring Boot and Spring Cloud*. Apress. https://doi.org/10.1007/978-1-4842-4501-9

- Docker Inc. Docker documentation. Docker Docs. https://docs.docker.com/

- Spring Boot Documentation
  VMware. (2025). *Spring Boot documentation*. Spring. https://spring.io/projects/spring-boot

2 Functional Requirements

2.1 User Requirements Specification

User Registration and Authentication

- Users should be able to register as public Attendees through a secure registration form that captures full name, email, role, and password.
- Registration should require personal information such as full name, email, and a secure password satisfying minimum policy rules.
- Public registration should be restricted to participant-level accounts in order to protect privileged roles.
- Users must be able to log in securely through an authentication system that returns a valid access token.
- The system should provide validation feedback for duplicate emails, invalid credentials, and missing mandatory fields.

User Profiles

- Attendees should have a profile showing their full name, email, role, and account status.
- Organizers should have a profile view with their identity details and role-specific access context.
- Admins should have a profile with visibility suitable for supervision and account management workflows.
- Users should be able to retrieve their own account data and, where permitted, update editable details.

Event Browsing and Participation

- Attendees should be able to browse upcoming events and inspect their main details before registration.
- Users should be able to open event detail pages showing description, venue, schedule, status, and seat availability.
- Attendees should be able to register for events if the event status permits it and seats are available.
- Attendees should be able to cancel a previous registration and see the effect reflected in their account data.
- Confirmed registrations should appear in a ticket-style page for quick access.

Organizer and Administrative Management

- Organizers should be able to view only the events assigned to them and inspect participant totals.
- Admins should be able to create, update, cancel, and reschedule events from an administrative dashboard.
- Admins should be able to create organizer or participant accounts and assign organizers to events.
- Notifications should inform relevant users about registrations, cancellations, assignments, and schedule changes.

2.2 System Requirements Specification.

Platform Infrastructure

- The system should be hosted on a reliable and scalable multi-service environment to ensure availability and performance.
- The platform should be developed using web technologies such as React, JavaScript, Java, Spring Boot, and a relational database such as PostgreSQL.
- The system should use an API Gateway as the unified entry point to backend services.
- Centralized configuration and service discovery should be used to support microservice communication.

User Authentication and Security

- Secure user authentication must be implemented for all protected roles.
- Passwords and sensitive user data must be protected using strong storage and transport practices.
- Role-based access control must be enforced to prevent unauthorized actions.
- Trusted internal notification endpoints must require an internal API key or equivalent trust mechanism.

Database Management

- A structured database schema will store user profiles, roles, events, registrations, and notifications.
- Data relationships should be normalized and indexed sufficiently for expected project workloads.
- Status changes should preserve historical records rather than relying on destructive deletion.

Search and Filter Functionality

- Users should be able to search or browse events using meaningful criteria such as title, date, or location.
- Admins should be able to filter operational views for users or events when managing platform data.

User Interface and Experience

- The platform should have a responsive and user-friendly interface accessible via desktop and mobile browsers.
- All users should have role-appropriate dashboards or main pages that summarize their key actions and history.
- Accessibility-conscious design choices such as clear contrast, readable labels, and understandable messages should be followed.

2.3 Requirements' Priorities

Using MoSCoW Scheme

Must Have

- Secure user registration and authentication
- User roles: Admin, Organizer, Attendee
- Event creation and event management features
- Event browsing and searching support
- Event registration and registration cancellation
- System-based notifications
- Admin dashboard for managing users and events

Should Have

- Ticket-style personal registration view
- Organizer-specific event monitoring page
- Managed-user creation from admin interface
- Availability display and participant count visibility
- Clear validation and error handling across all major workflows

Could Have

- Printable or downloadable tickets
- Email or SMS notification integration
- Analytics dashboards for event trends
- Export Calendar
- Advanced recommendations or smart search features

Won't Have

- Integration with third-party payment gateways (e.g. PayPal) in the current version
- Real-time chat support in the current version
- Multilingual support beyond English in the current version

3 Non-Functional Requirements

3.1 The General Types/Categories of Non-Functional Requirements

The system adheres to the following NFR categories, aligned with software engineering quality classifications:

1. **Product Requirements:** reliability, performance, usability, security, maintainability, and compatibility.
2. **Organizational Requirements:** modular implementation, project documentation, deployment consistency, and testability.
3. **External Requirements:** privacy expectations, browser compatibility, and trust-preserving system behavior.

3.2 Non-functional requirements Specification

Performance Requirements

- Common dashboard and event-listing requests should complete within a few seconds under normal academic-project load.
- The database and services should handle expected concurrent read and write activity for class- project use without severe degradation.
- Availability and registration calculations should remain consistent when multiple users act close together in time.

Security Requirements

- All protected user authentication should use secure password storage and token validation.
- Role-based access control should restrict administrative functions to authorized personnel only.
- Internal service actions must remain protected from arbitrary public invocation.

Usability Requirements

- First-time attendees should be able to register for an event without special training.
- Error messages should provide clear resolution instructions in plain language.
- Important actions such as Register, Cancel Registration, Assign Organizer, and Mark as Read should be easy to understand.

Reliability Requirements

- Persistent events, registration, notification, and user data should survive service restarts.
- Historical data should be retained through status transitions.

Scalability Requirements

- The architecture should support growth in user count and event volume without full redesign.
- Individual services should be improved independently because of microservice separation. Availability Requirements
- Critical events and registration functions should remain available during minor updates when supporting infrastructure is healthy.

Maintainability Requirements

- Code documentation and service separation should cover core modules sufficiently for team maintenance.

Compatibility Requirements

- The interface shall be rendered correctly on Chrome, Firefox, and Edge.

3.3 The fit criteria for every Non-Functional Requirement

Performance Verification

- Load testing with simulated users executing login, event browsing, registration, and notification retrieval workflows.
- Database and API profiling during peak

usage scenarios.

Security Verification

- Manual code review of authentication, authorization, and internal API protection logic.
- Endpoint testing for unauthorized access attempts. Usability Verification
- Task completion metrics from user walkthrough sessions.
- Interface review of labels, messages, and screen flow clarity.

Reliability Verification

- Restart tests to confirm persistence of accounts, events, registrations, and notifications.
- Status-based workflow review for safe historical retention.

Scalability Verification

- Stress testing with gradually increasing user loads.
- Architecture review for service independence and isolated optimization potential. Maintainability Verification
- Static structure review of service boundaries, module organization, and code readability.
- Documentation consistency checks between implementation and specification.

3.4 How would the above-mentioned Non-Functional Requirements affect the System's overall Architecture?

The non-functional requirements directly shape the system architecture

- ❖ Performance demands lead to separate services, targeted API responsibilities, and indexed data access for events and registrations.
- ❖ Security requirements necessitate layered defense including authentication, authorization, token validation, and internal API protection.
- ❖ Reliability needs drive status-based history retention and isolated service behavior.
- ❖ Scalability considerations favor modular microservice design with clear interfaces between domains.
- ❖ Usability goals require role-focused pages and a frontend that hides backend complexity from the user.

4 Design & Implementation Constraints.

Technology Stack

- **Frontend**: React, Vite, JavaScript, CSS utility styling, and Axios for API communication.
- **Backend**: Java and Spring Boot for service development and business logic handling.
- **Database**: PostgreSQL, a relational database management system, used to store and manage structured data securely.
- **Infrastructure**: Spring Cloud Gateway, Spring Cloud Config, Eureka Server, Docker, and Docker Compose.

Compatibility Requirements

- The system must be fully compatible with major desktop browsers, including Google Chrome, Mozilla Firefox, and Microsoft Edge.

Hosting Environment

- The platform is intended for container-friendly execution in a local or cloud-capable environment capable of running multiple Java services and relational databases.
- Configuration values should remain externalized through the configuration server and deployment files.

Project Constraints

- Time & Resources: the development team operates under academic project deadlines and limited team resources.
- Budget: financial limitations may reduce access to premium tooling, hosted services, and advanced integrations.
- Technical Limitations: microservice orchestration, environment setup, and cross-service debugging introduce additional project complexity.

5 System Evolution

5.1 Anticipated changes

- Integration with email and SMS delivery providers for external notifications.
- Implementation of paid ticket support and third-party payment integration in the future phase.
- Development of a dedicated mobile application or progressive web app experience.
- Potential migration to more advanced monitoring, caching, or cloud-native deployment patterns for enhanced scalability.
- Addition of analytics, recommendation, or event check-in modules.

5.2 How should any anticipated changes in the future (due to hardware evolution, changing user needs, and so on) affect the system design?

The Event Registration System should be designed with flexible, modular, and future-proof architecture to support anticipated growth and evolving user needs. **To ensure long-term adaptability**, the system should remain accessible across multiple device types, compatible with different deployment environments, and open to future integration with external services such as payments, communications, analytics, and attendance tracking tools.

- A modular design that separates presentation, gateway routing, domain services, and persistence concerns.
- Standard APIs and RESTful communication for interoperability with future subsystems.
- Clean and well-documented code to facilitate onboarding of new developers and safe future enhancement.

6 What are the requirements discovery approaches that you'll rely on?

To ensure the development of a system that meets the actual needs of users and stakeholders, we rely on several effective discovery techniques:

Interviews and Local Surveys

We collected and inferred expectations from likely users such as students, organizers, and coordinators. These discussions help highlight practical needs such as easy event browsing, quick registration, schedule visibility, and administrative control.

Direct Observation

We observed how event participation is usually handled in academic or community settings, including spreadsheet lists, messages, and late manual updates. This approach provided a practical understanding of real-world event workflows and challenges.

Similar System Review

We reviewed common behaviors in event and registration platforms to identify expected features such as event cards, ticket confirmation, notifications, and role-specific dashboards.

Codebase-driven Discovery

Because this project is already implemented, the current codebase also serves as a discovery source. Controllers, services, entities, and frontend pages reveal the precise operations already supported and therefore must be reflected in the formal SRS.

7 What requirements validation techniques will you employ/use?

Requirements Reviews (Clarifying & Restating Requirements)

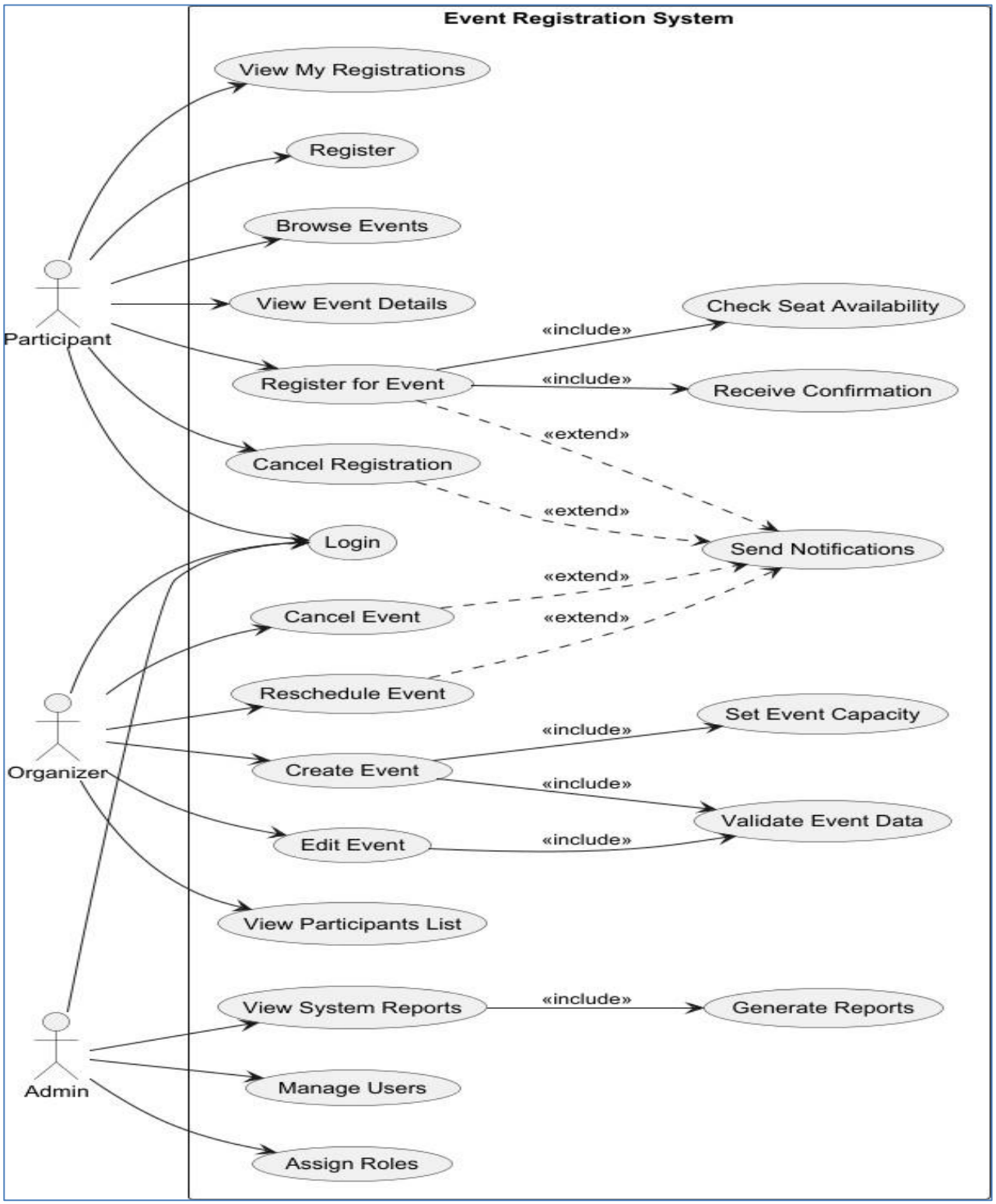
- Review the written requirements with team members or stakeholders to confirm that they are understandable by all and aligned with actual project goals.
- Check that each requirement is specific enough to guide implementation and testing.
- Validate that role permissions and workflow restrictions reflect the system's real intended behavior.

Prototyping & Walkthroughs (Validating Requirements)

- Walk through the event browsing, registration, admin event management, and organizer assignment flows with sample users.
- Use the frontend itself as a validation artifact for realistic user interaction checking. Implementation Traceability
- Map requirements to implemented controllers, services, entities, and frontend pages.
- Use this mapping to detect missing, overstated, or weakly documented functionality.

PART 2: Overview & Software Requirements Specification

8 Functional Diagrams: Use-Case Diagram including all the system Use Cases



Detailed Use-Cases Description

ID	Name	Goal	Initiator	Pre-condition(s)	Post-condition(s)	Main Success Scenario	Alternate Scenario	Exception Scenario
1	Register account	Create participant account	Visitor	Visitor is on registration page	Account created	Open register page -> Enter data -> Submit -> System validates -> Account saved	Visitor closes form before saving	Duplicate email or weak password
2	Log in	Access role-based features	All roles	Account exists	User enters system	Open login -> Enter email and password -> Token issued -> Redirect by role	User stays on login page	Invalid credentials
3	View profile	See account information	All roles	User authenticated	Profile displayed	Open profile -> System loads full name, email, role, status	User refreshes page	Session invalid
4	Browse events	See all events	Visitor / participant	Platform available	Event list shown	Open events page -> Fetch events -> Render cards	Filter by title/date	Event service unavailable
5	View event details	Inspect one event	All users	Event exists	Details shown	Select event -> Open details -> Load description, venue, seats, schedule	Return to list	Event not found
6	Register for event	Reserve event seat	Participant	Logged in, seats available	Registration created	Open event -> Click register -> Validate role/status/seats -> Save registration -> Notify user	User changes mind before submit	Event full or cancelled

I D	Name	Goal	Initiator	Pre- condition(s)	Post-condition(s)	Main Success Scenario	Alternate Scenario	Exception Scenario
7	Cancel registration	Withdraw attendance	Participant	Active registration exists	Registration cancelled	Open my tickets -> Choose cancel -> Confirm -> Status changed -> Notification generated	User keeps registration	Already cancelled
8	View my registrations	See confirmed registrations	Participant	At least zero registrations	Ticket list displayed	Open tickets page -> Load confirmed registrations only	User follows event link	Registration list load failure
9	Create event	Add new event	Admin	Authenticated admin	Event stored	Open dashboard -> Fill form -> Validate fields -> Save event -> Refresh list	Admin saves draft mentally and closes page	Missing organizer or invalid date
10	Edit event	Modify event data	Admin	Event exists and not cancelled	Event updated	Select event -> Edit fields -> Submit -> Validate -> Save	Admin opens edit and exits	Cancelled event cannot be edited
11	Cancel event	Mark event cancelled	Admin	Event exists	Status becomes cancelled	Select event -> Cancel -> Save cancelled state -> Notify affected users	Admin delays action	Event already cancelled
12	Reschedule event	Change event timing	Admin	Event exists and editable	Status becomes rescheduled	Select reschedule -> Enter new dates -> Validate -> Save -> Notify users	Admin closes reschedule panel	End time before start time
13	Assign organizers	Set organizer ownership	Admin	Event exists	Organizer list updated	Open assign tab -> Select event -> Select organizers -> Save assignment	Admin reviews list only	No organizer selected

ID	Name	Goal	Initiator	Pre-condition (s)	Post-condition(s)	Main Success Scenario	Alternat e Scenario	Exception Scenario
14	Receive Confirmation	Confirm registration success to the user	System	Registration process is successful	User receives a digital receipt/ confirmation	Registrati on is complete. 2. System displays confirmati on message	System shows confirmatio n inside app instead of email	Confirmatio n on email fails to send
15	View event participants	See registrations for one event	Admin / Organizer	Authorized user, event exists	Participan t list displayed	Request event registration s -> Vali date organiz er scope -> Return rows	User refreshes list	Unauthorize d access
16	Create managed user	Create organizer/parti ci pant account	Admin	Admin logged in	Account created	Open users tab -> Enter user data -> Validate -> Save account	Admin clears form	Email used or role invalid
17	Create notification	Store notification	Admin / self user	Authenticate d	Notification created	Submit type/title/me s sage -> Validate -> Save unread notification	Draft not submitted	Missing target user
18	Set event capacity	Define the maximum number of seats	Organizer	Creating or editing an event	Max capacity is set	Organi zer enters seat number->System stores capacity limit	Organizer changes capacity → system updates seats	Negative number entered; system rejects input
19	Validate token	Check current session	Frontend client	Token exists	Validation response returned	Client calls validate endpoint -> Service checks token principal -> Return validit y	Token absent	Malformed token

20	Check set availability	Check remaining seats	Client / service	Event exists	Availability returned	Call availability endpoint -> Read registered count -> Compute remaining seats	Repeated checks	Downstream registration service error
21	Generate Reports	Create visual/textual data for admin	System	System data is available	Report file/view created	System aggregates data-> Generates dashboard/PDF	Admin chooses filters → system shows filtered report	Data timeout; partial report generated

9 Structural & Behavioral Diagrams

9.1 System Architecture

The Event Registration System is a distributed platform designed to manage event publication, secure access, participant registration, organizer coordination, and administrative oversight**. Microservices + Gateway + Role-Aware Frontend architectural style** is ideally suited for this project because it aligns with the following requirements and characteristics

Separated Domain Responsibilities

The platform includes authentication, event management, registration processing, and notifications. Splitting these into dedicated services keeps each domain focused and easier to maintain.

Role-Based User Interaction

The frontend exposes distinct experiences for Admins, Organizers, and Attendees, while backend authorization ensures protected operations remain role aware.

Workflow Orchestration

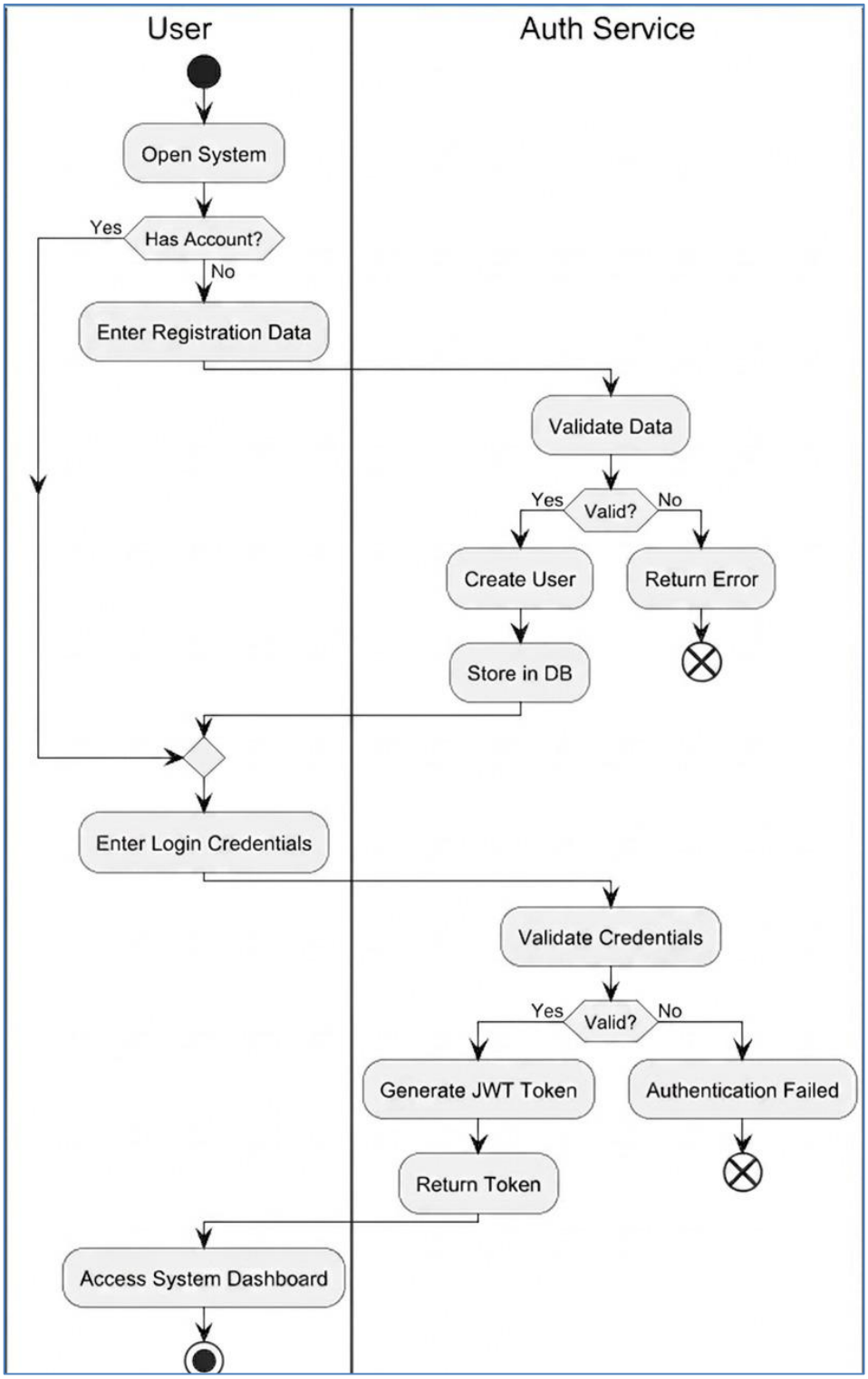
Frontend components call the API Gateway, which routes requests to service-specific controllers. Services coordinate through internal APIs where needed for availability checks and notifications.

Scalability and Future Extensibility

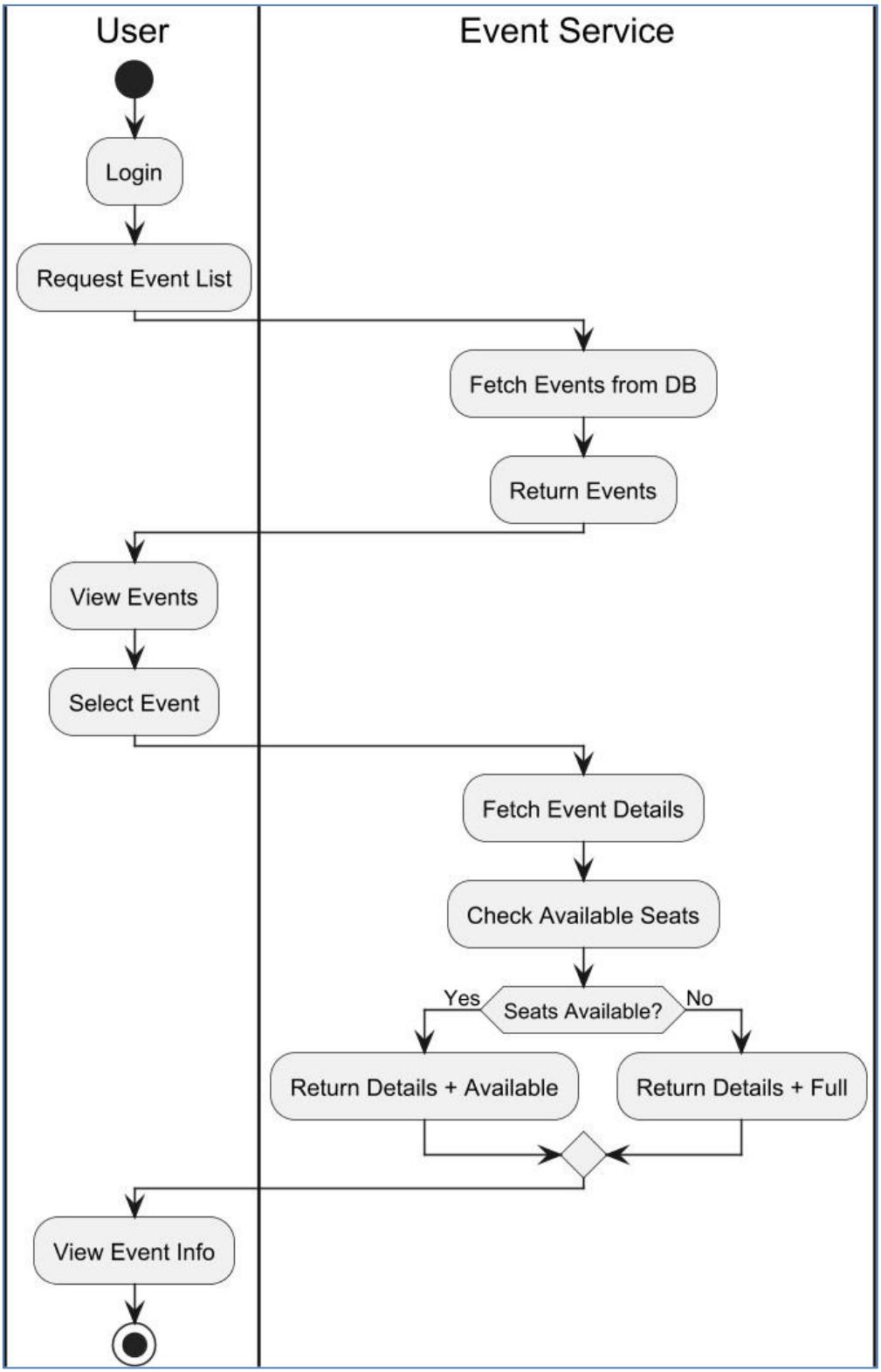
New features such as payments, attendance check-in, analytics, or external notifications can be introduced without collapsing all logic into one monolith.

Testability and Maintainability: service-level units and controller tests can be executed in isolation, reducing complexity during verification and change management.

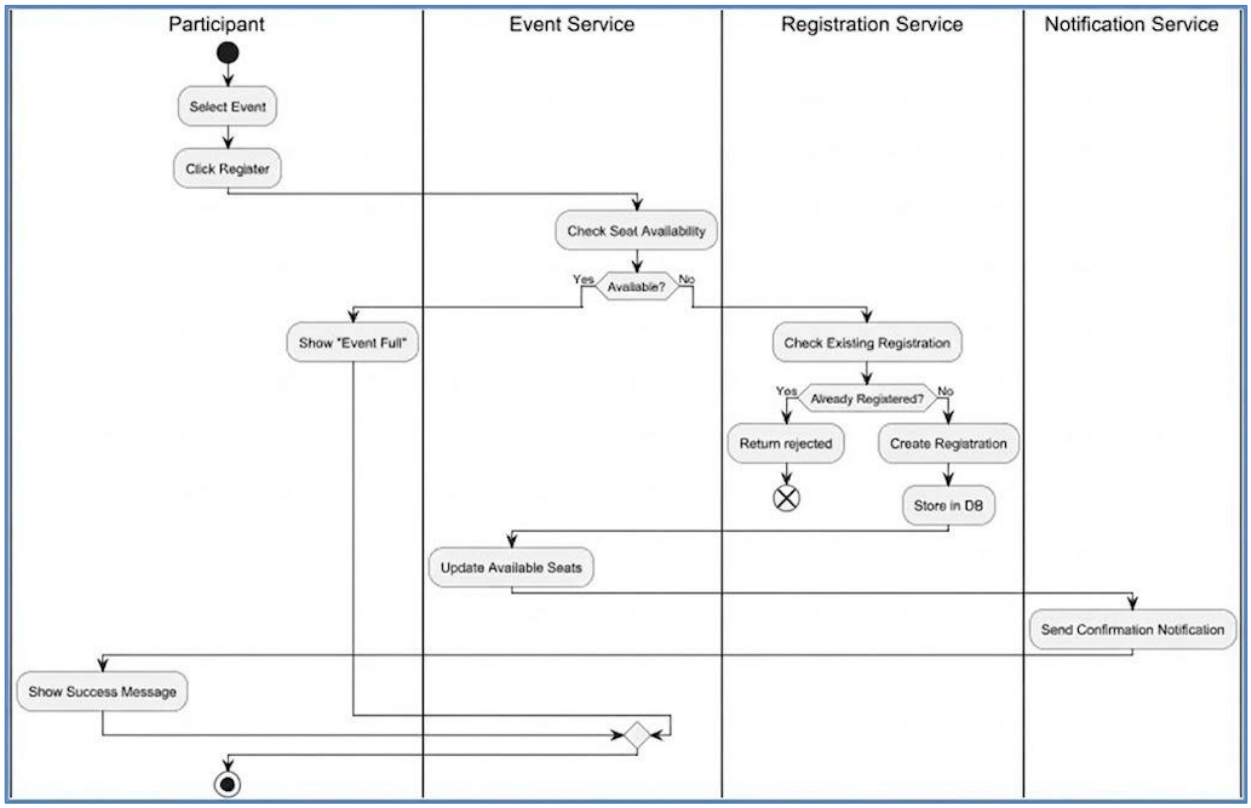
9.2 Activity Diagrams : Activity1 :Registration & Login Flow



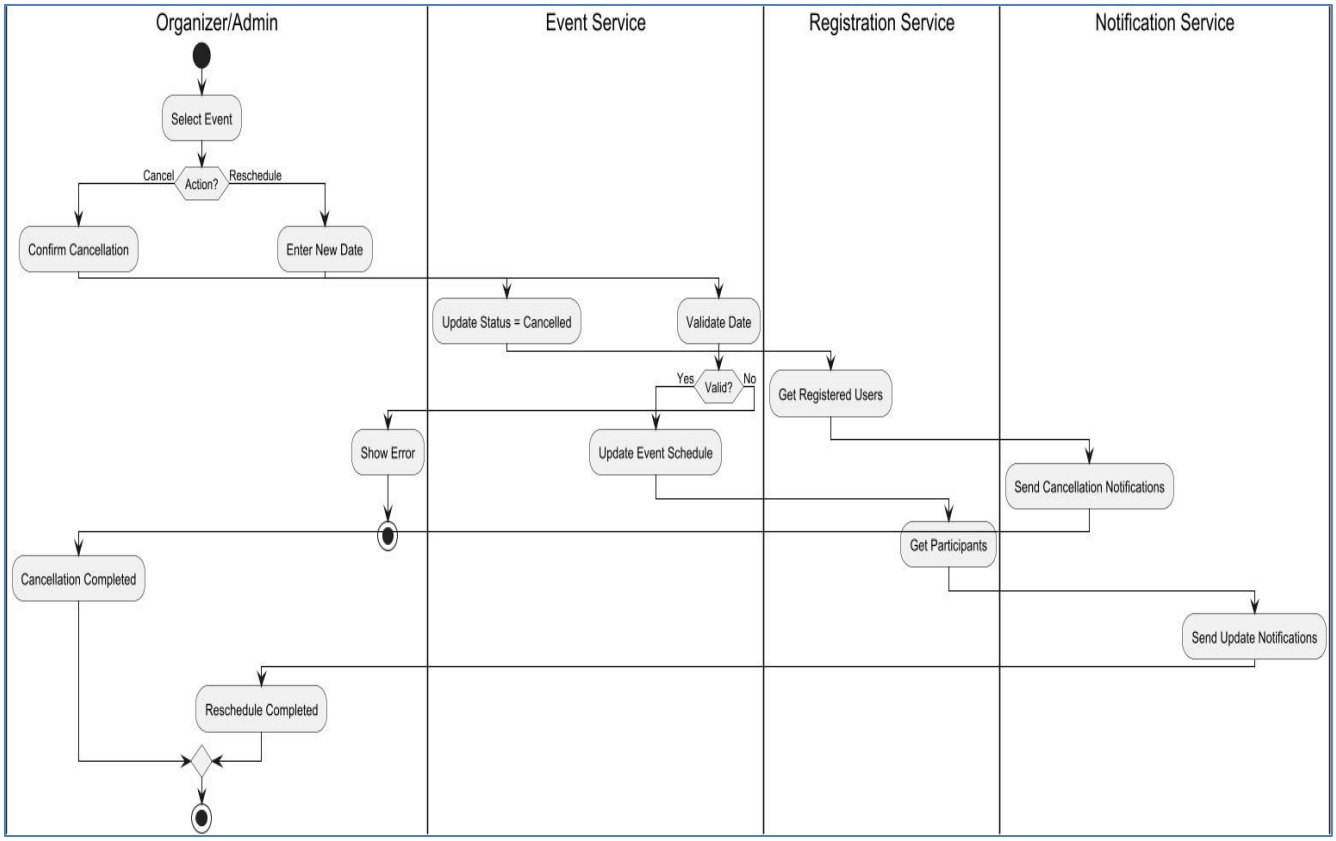
Activity2 : Event Browsing and Detail Viewing



Activity3 : Event Registration



Activity4 : Event Rescheduling / Cancellation and Notification Flow



9.3 Based on the Activity Diagrams, the List of User Interfaces required for the System and the corresponding users of each interface

1. Login & Registration

Functions

- User authentication (login/logout)
- New account registration for public participant users
- Token validation and protected-route entry

Users

- Attendees
- Admins
- Organizers

2. Event Discovery

Functions:

- Browse all events
- View event details
- Check event availability
- Trigger registration

Users

- Visitors
- Attendees

Participant Account Interface

Functions

- View profile
- View personal notifications
- View confirmed tickets
- Cancel registrations

Users

- Attendees

Organizer Dashboard Interface

Functions

- See assigned events
- Monitor participant counts
- Open event details
- Read organizer-relevant notifications

Users

- Organizers

Admin Panel Interface

Functions

- Event creation
- Event editing
- Event cancellation and rescheduling
- Managed-user creation
- Organizer assignment

Users

- Admins

Notification Interface
Functions

- Display read and unread notifications
- Mark notification as read

Users

- All authenticated users

Supportive Event Form Interface
Functions

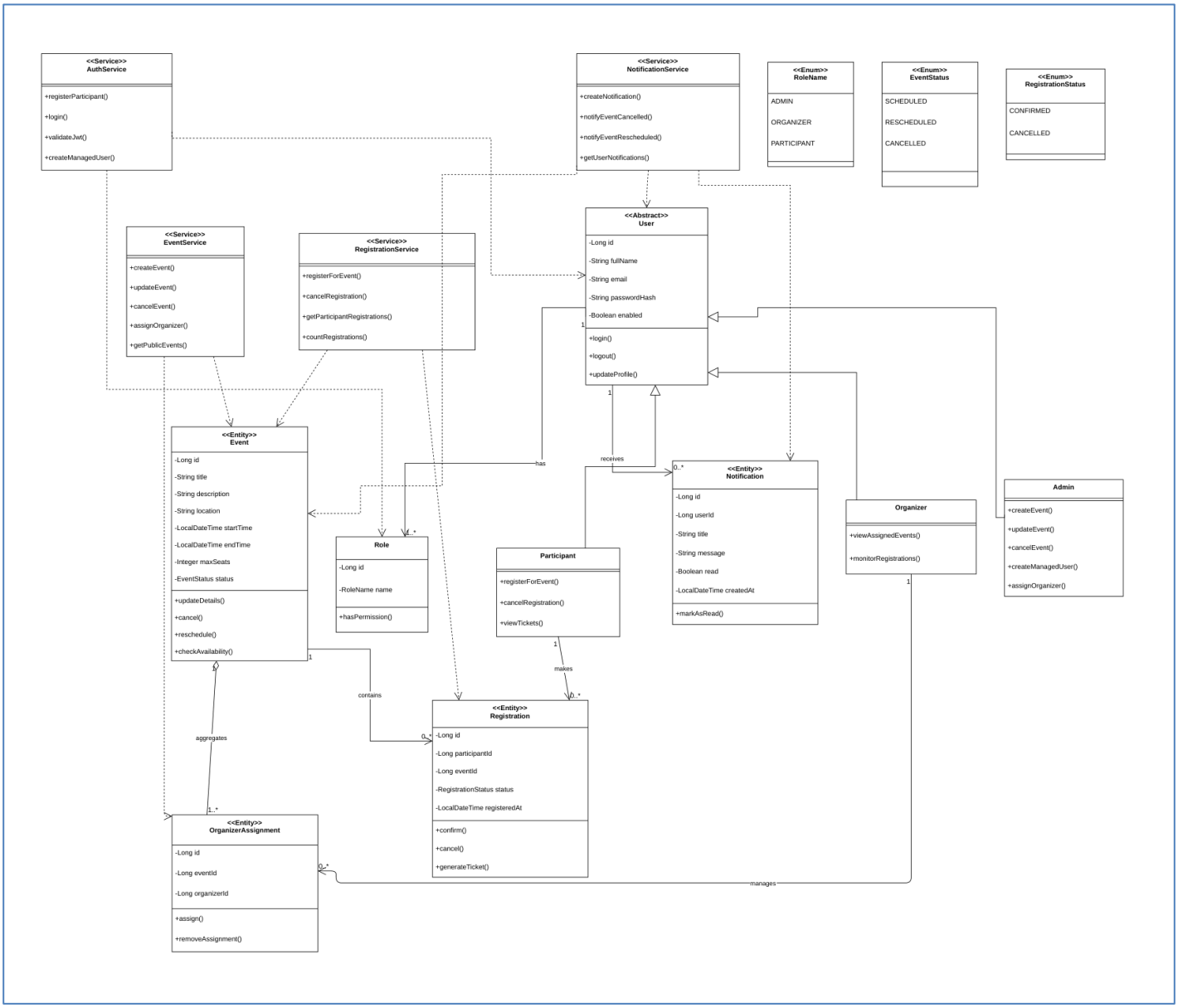
- Create or edit event title, description, location, schedule, capacity, and organizers

Users

- Admins

9.4 Class Diagram 1: An initial version based on the requirements and Use Case/Activity diagrams

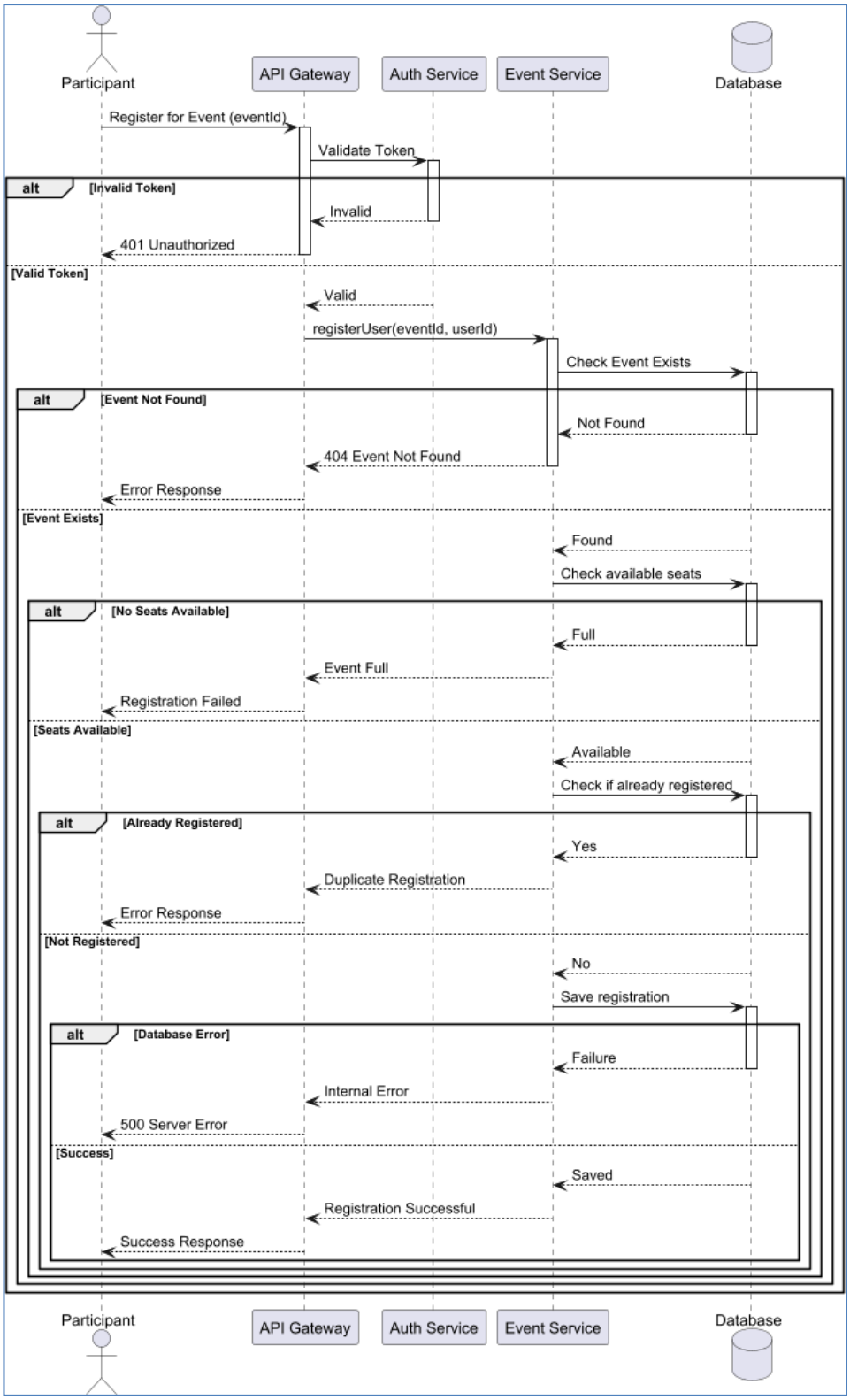
Initial domain classes include User, Role, Event, Registration, Notification, AuthSession representation, OrganizerAssignment view, EventAvailability view, and Dashboard-oriented boundary/control objects.



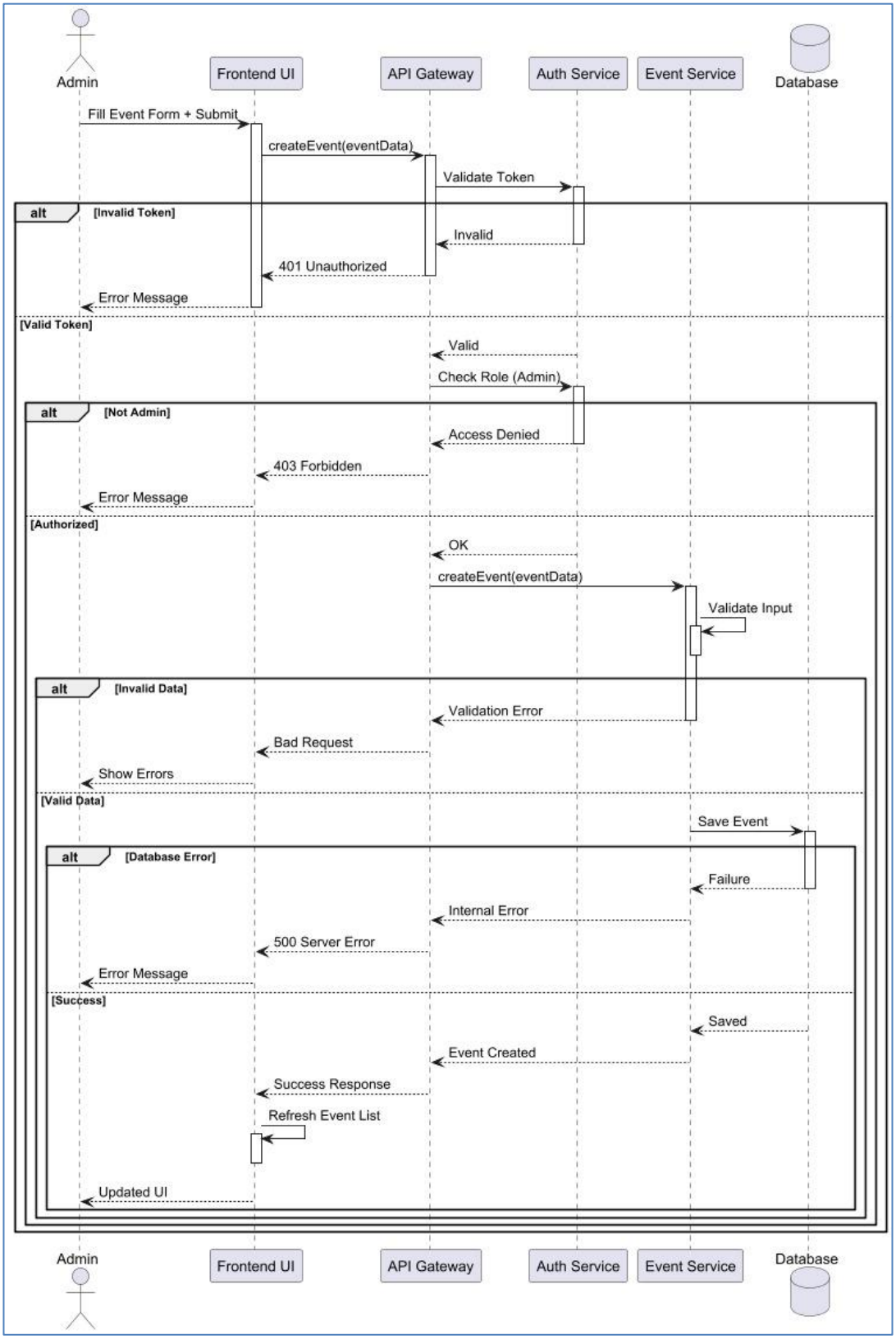
9.5 Sequence

Diagram(s): Sequence

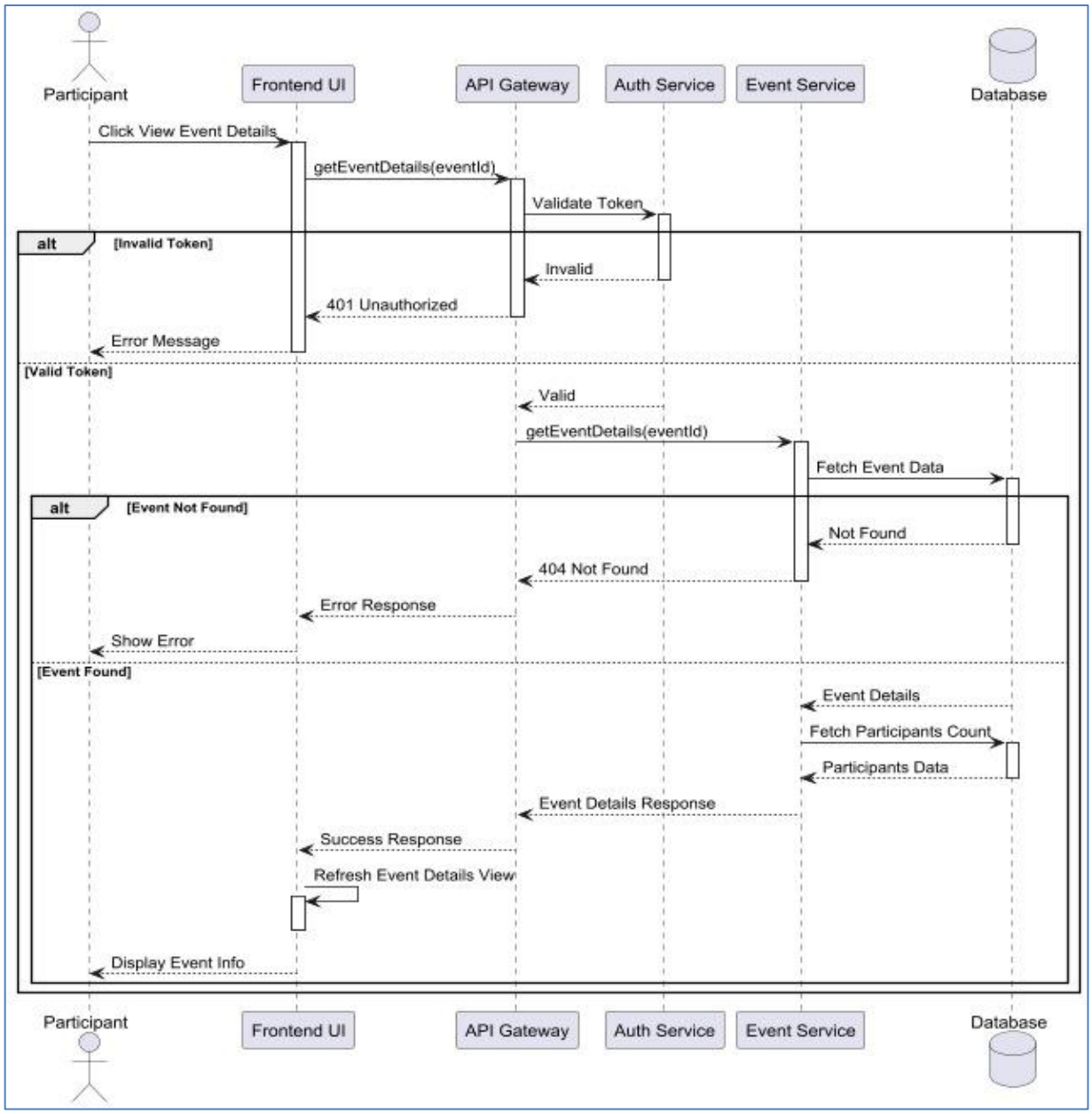
1: Register for Event



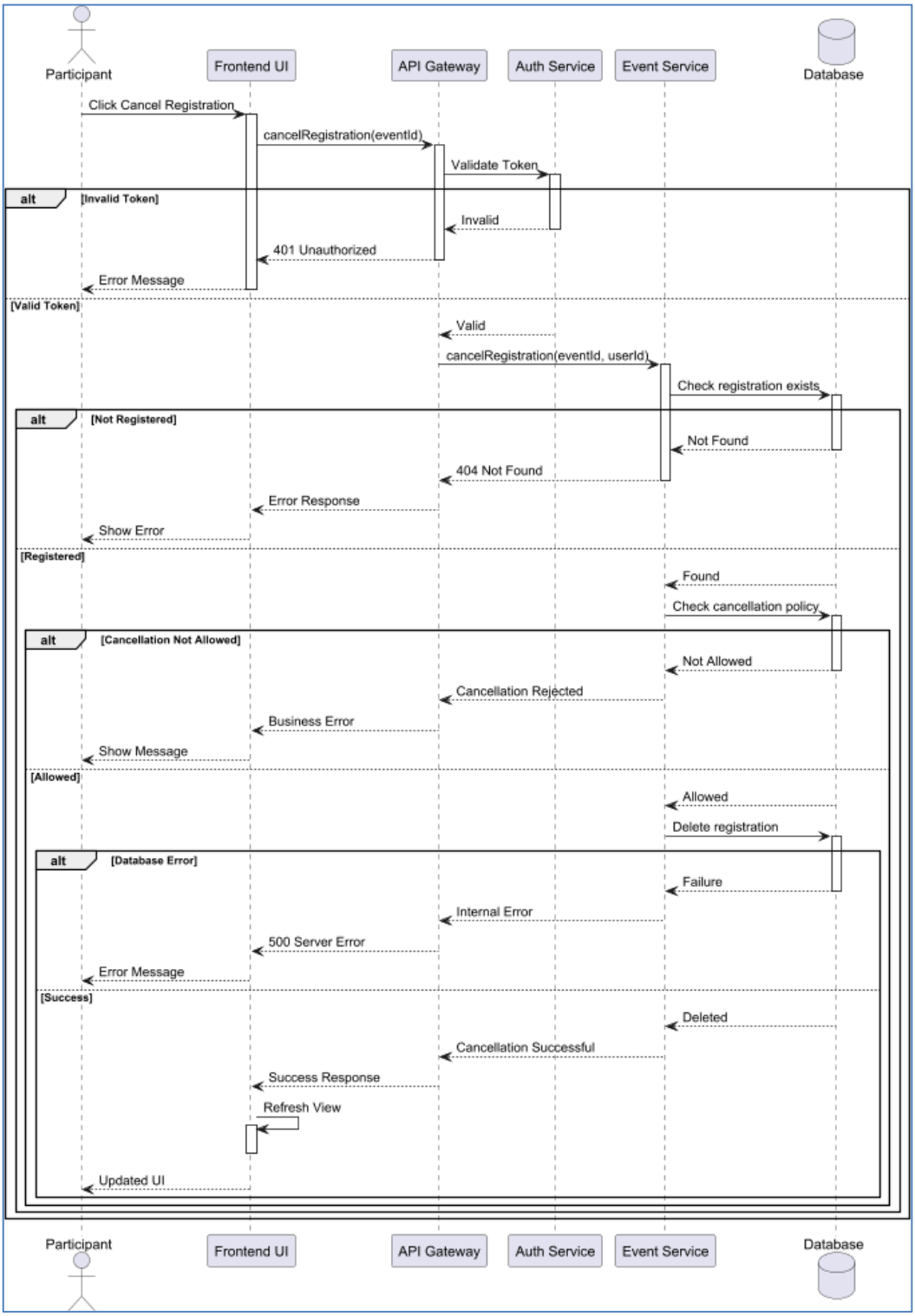
Sequence 2:Create Event



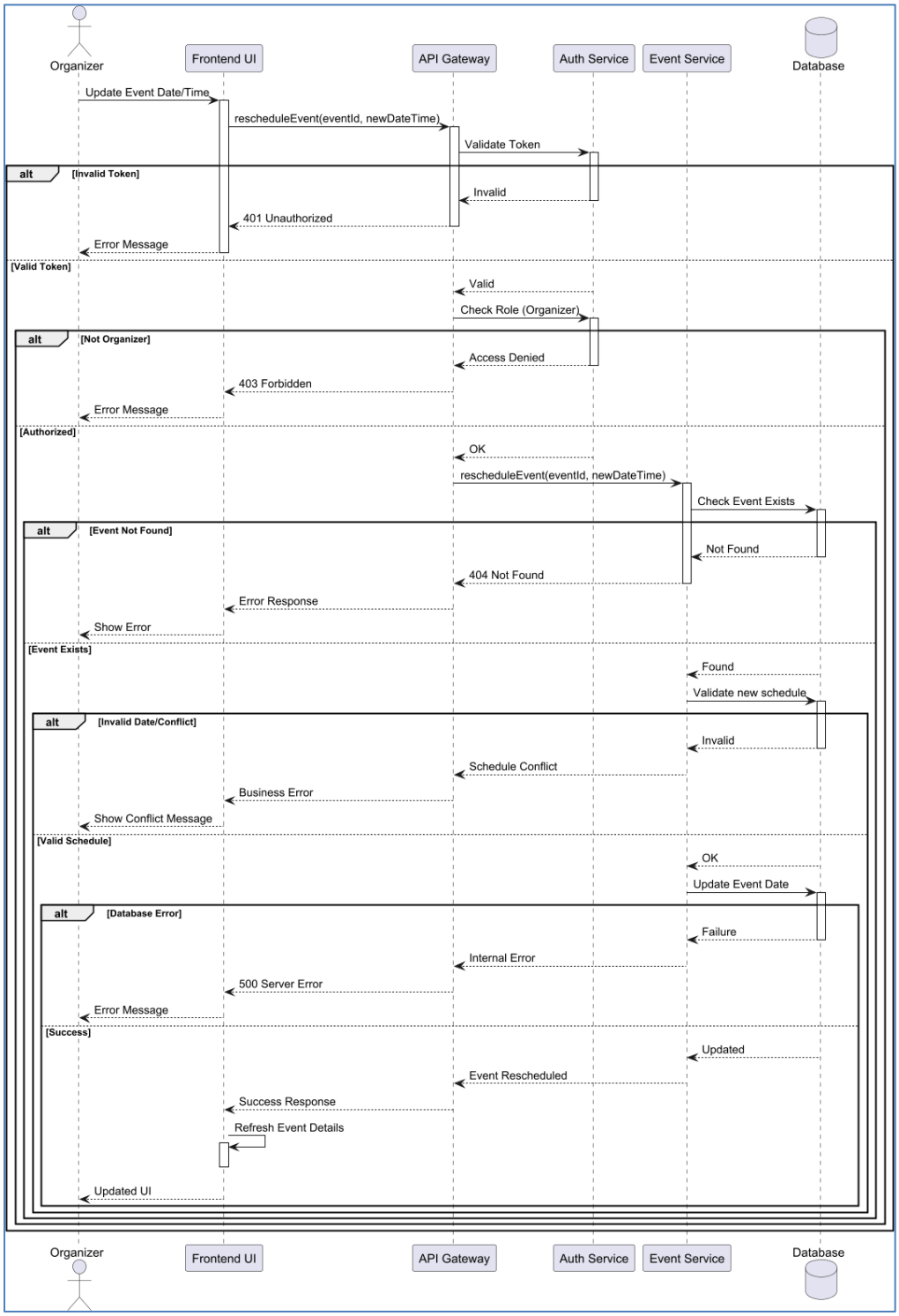
Sequence 3: View Event Details



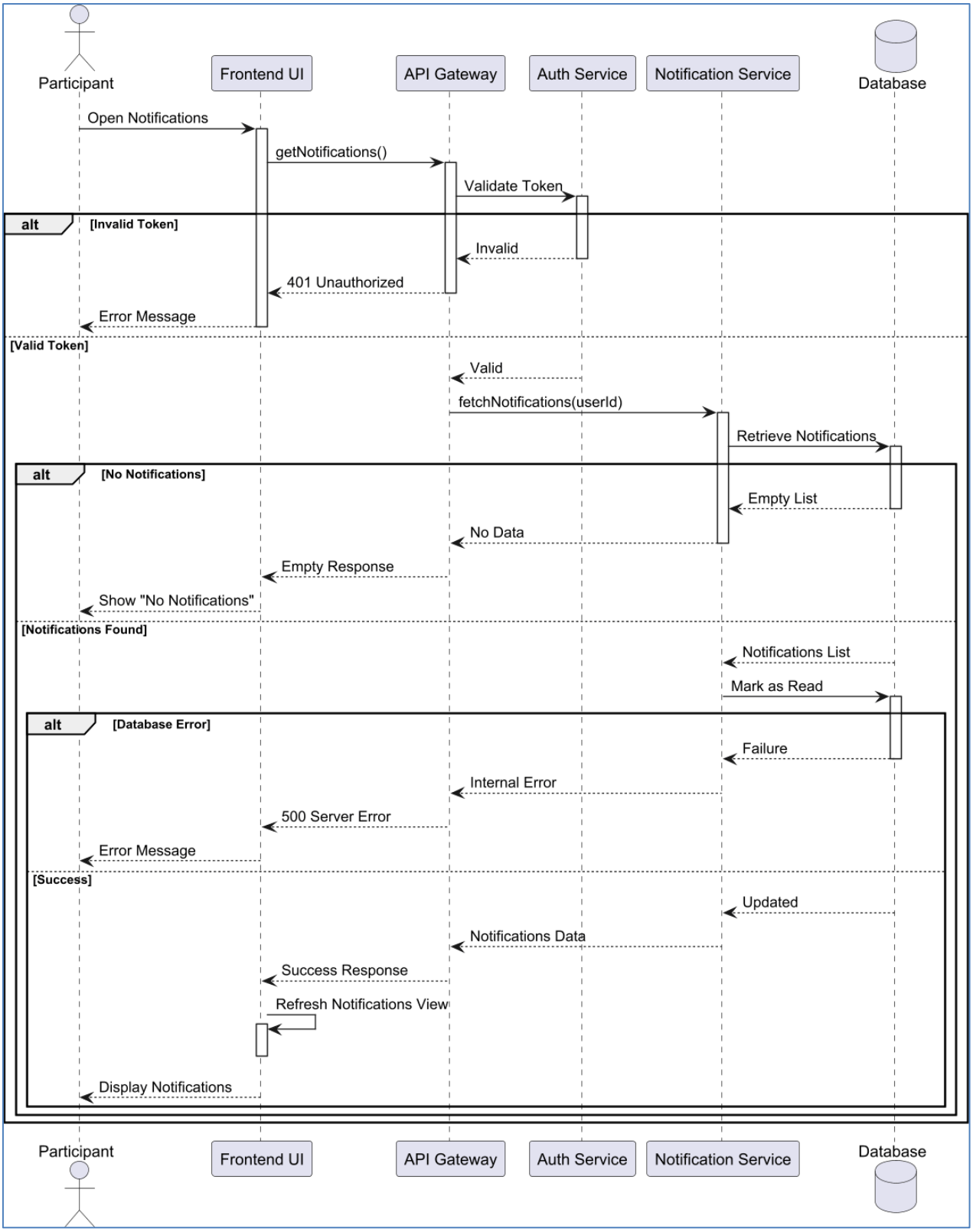
Sequence 4: Cancel Registration



Sequence 5: Reschedule Event



Sequence 6: Read Notifications



9.8 Which strategy (or strategies) did you use to implement the use cases: One Central Class, Actor Class, or Use-Case class?

The strategy used in the implemented Event Registration System is closer to a Use-Case Class combined with service-oriented control logic. We chose this strategy because each major business capability is represented by dedicated service classes such as AuthService, EventService, RegistrationService, and NotificationService, while controllers expose actor-accessible endpoints.

This approach was most suitable for our system because use-case logic is grouped around domain responsibilities rather than concentrated in one central class or duplicated directly in actor-specific classes.

Advantages

- Improves modularity by placing authentication, event, registration, and notification behavior in focused services.
- Makes business-rule testing easier because each service can be tested in isolation.
- Supports microservices architecture and clear API boundaries.
- Reduces tight coupling between frontend pages and backend implementation details.

Disadvantages

- Cross-service flows such as registration plus availability plus notification are more complex to trace.
- Additional infrastructure is needed for routing, discovery, and configuration.
- Operational debugging may become harder compared to a small monolithic project.

9.9 Design Patterns Applied in the Event Registration System

This section describes the design patterns and architectural patterns used in the Event Registration System. Each pattern is linked to its purpose and its concrete implementation within the project structure.

1. Layered Architecture (MVC Style)

The system follows a layered architecture where responsibilities are separated into presentation, business logic, and data access layers.

Where It Appears

- controller package (e.g., AuthController, EventController, RegistrationController)
- service package (e.g., AuthService, EventService)
- repository package (e.g., UserRepository, EventRepository)
- dto package for request/response models

Purpose

- Separates concerns between request handling, business logic, and database access
- Improves maintainability and testability
- Makes the system easier to scale and modify

2. Repository Pattern

Spring Data repositories are used to abstract database operations from the service layer.

Where It Appears

- UserRepository
- RoleRepository
- EventRepository

- RegistrationRepository
- NotificationRepository

Purpose

- Encapsulates data access logic
- Eliminates direct database queries in service layer
- Improves code readability and flexibility

3. DTO (Data Transfer Object) Pattern

DTOs are used to transfer data between client and server instead of exposing entity models.

Where It Appears

- dto/request packages
- dto/response packages across all services

Purpose

- Protects internal data structure from external exposure
- Controls API response structure
- Supports validation and clean API design

4. Dependency Injection

Dependencies are injected automatically by Spring Framework through constructors.

Where It Appears

- Controllers injecting services
- Services injecting repositories and external clients
- Security and AOP components managed by Spring container

Purpose

- Reduces tight coupling between classes
- Improves testability
- Delegates object creation to Spring container

5. API Gateway Pattern

An API Gateway is used as a single entry point for all client requests.

Where It Appears

- api-gateway service configuration
- Routing rules in application.yml

Purpose

- Provides a unified entry point for frontend clients
- Routes requests to appropriate microservices
- Simplifies client-side communication

6. Service Discovery Pattern

Services register themselves with a Eureka Server for dynamic discovery.

Where It Appears

- Eureka Server module
- Service configuration files referencing Eureka URL

Purpose

- Removes need for hardcoded service URLs
- Supports scalability and dynamic scaling of services
- Enables inter-service communication in distributed systems

7. Centralized Configuration Pattern

Spring Cloud Config Server is used to manage configuration in a centralized manner.

Where It Appears

- config-server
- Service-specific configuration files (auth-service.yml, event-service.yml, etc.)

Purpose

- Centralizes configuration management
- Supports environment-based configuration (dev, prod, etc.)
- Simplifies configuration updates across services

8. Client / Proxy Pattern

Inter-service communication is handled through dedicated client classes.

Where It Appears

- NotificationServiceClient
- RegistrationServiceClient
- EventServiceClient
- UserServiceClient

Purpose

- Encapsulates remote service calls
- Reduces duplication of HTTP communication logic
- Improves maintainability of service integrations

.

Cloud Deployment model

Overview

The Event Registration System is designed as a cloud-native, microservices-based application. The system is fully containerized using Docker and is deployable on cloud-based virtual machines or Kubernetes clusters without requiring any modifications to the core application logic. This design ensures portability, scalability, and ease of deployment across different cloud environments.

Cloud Readiness Justification

The system is considered cloud-ready due to its adoption of established cloud-native principles and supporting technologies, including:

- A microservices architecture that enables independent development, deployment, and scaling of services
- An API Gateway acting as a single entry point for all external client requests
- Service discovery implemented via Eureka Server to support dynamic service registration and communication
- Centralized configuration management using Spring Cloud Config Server
- Database-per-service design using isolated PostgreSQL instances for improved data separation and scalability
- Full containerization of all backend services using Docker
- Docker Compose support for simplified deployment in local and virtual machine environments
- Kubernetes manifests enabling orchestration, scaling, and high availability in production-grade environments

System Deployment Architecture

The system follows a layered cloud deployment architecture defined as:

Client → API Gateway → Microservices Layer

Entry Point

- **API Gateway** (Port: 8080)
  Serves as the unified external interface and routes all incoming client requests to the appropriate backend services.

Backend Microservices

- **Auth Service** (Port: 8081) – Responsible for authentication and authorization
- **Event Service** (Port: 8082) – Manages event-related operations
- **Registration Service** (Port: 8083) – Handles user registrations for events
- **Notification Service** (Port: 8084) – Manages system notifications and alerts

Supporting Infrastructure Components

- **Config Server** (Port: 8888) – Provides centralized configuration management
- **Eureka Server** (Port: 8761) – Enables service discovery and registration
- **PostgreSQL Database** (Port: 5432) – Provides persistent data storage

In this architecture, all external communication is strictly routed through the API Gateway, while inter-service communication is managed internally within the microservices ecosystem.

Deployment Strategies

Docker Compose Deployment (Cloud Virtual Machine)

This deployment approach is suitable for cloud-based virtual machines such as AWS EC2, Microsoft Azure Virtual Machines, or Google Compute Engine instances.
Deployment command:
docker compose up -d –build

Key characteristics

- All services are deployed as independent containers
- The API Gateway is the only externally exposed component
- Internal services communicate through a private Docker network, ensuring isolation and security

Kubernetes Deployment

This approach is intended for production-grade environments requiring scalability, resilience, and orchestration capabilities.

Deployment command
kubectl apply -f cloud/kubernetes/event-registration-system.yml

Verification commands
kubectl get pods
kubectl get services

For local testing using Minikube
minikube service api-gateway

In production environments

- The API Gateway is exposed via a LoadBalancer service or Ingress controller
- Container images are stored in a container registry prior to deployment
- Kubernetes manages scaling, fault tolerance, and service orchestration

Cloud-Native Design Principles

The system adheres to key cloud-native architecture principles, including:

- Decoupled microservices enabling independent deployment and scalability
- Centralized routing and request management through an API Gateway
- Dynamic service discovery using Eureka Server
- Externalized configuration management via Spring Cloud Config Server
- Containerized deployment ensuring environment consistency
- Kubernetes compatibility for orchestration and scaling in distributed environments

Production Deployment Considerations

For production-grade deployment, the following best practices are recommended:

- Restrict external access exclusively to the API Gateway
- Secure sensitive configuration using environment variables or dedicated secret management solutions
- Implement persistent storage mechanisms for PostgreSQL databases
- Enable HTTPS communication through a reverse proxy, Ingress controller, or cloud-native load balancer
- Integrate centralized logging, monitoring, and health-checking mechanisms across all services
- Utilize container registries with versioned images to ensure deployment consistency and rollback capability
- Enable horizontal scaling of microservices based on system load and performance requirements

9.10 Define a design smell, highlight a design smell in your design, and suggest how it can be avoided.

Definition: What Is a Design Smell?

A design smell is a weakness or flaw in the architecture of a software system that suggests poor design decisions. While not necessarily causing bugs immediately, it often leads to low cohesion, high coupling, difficult testing, or future maintenance pain.

Highlighted Design Smell: "God Service" Risk in EventService

One possible design smell in our Event Registration System is the risk that EventService becomes too responsible over time. It already handles creation, update, cancellation, rescheduling, organizer assignment, availability composition, and notification triggering. If future features such as analytics, ticket generation, or reporting are added directly into the same service, it may evolve into a God Service.

How We Avoid This Smell

- Split responsibilities into focused services such as EventSchedulingService, OrganizerAssignmentService, or EventReportingService if complexity grows.
- Use dedicated client or notification coordination classes for cross-service communication.
- Follow SOLID principles, especially SRP and OCP, when introducing new event-related features.

The general categories are an Entity Class, a Boundary Class, a Control Class, or an Application Logic Class.

Entity Classes

- User, Role, Event, Registration, Notification

Boundary Classes

- LoginPage, RegisterPage, FindEventsPage, EventDetailsPage, MyTicketsPage, NotificationsPage, OrganizerPage, AdminDashboardPage, ProfilePage

Control Classes

- AuthController, EventController, RegistrationController, RegistrationQueryController, NotificationController

Application Logic Classes

- AuthService, EventService, RegistrationService, NotificationService, JwtService, EventLockManager, client adapters, security filters, and configuration services

9.11 Did you rely on Forks or Cascades in your interaction diagrams?

In the interaction design of our Event Registration System, we primarily adopted the Cascade structure. This choice was intentional because many of our workflows require ordered validation and state transition. For example, an event registration request must authenticate the user, validate the participant role, verify event status, confirm seat availability, store the registration, and then trigger notifications. This sequence is naturally cascade-oriented.

The Advantages

- Clear visualization of sequential validation and persistence flow.
- Easy to follow for academic documentation and system explanation.
- Helps identify dependencies between services and validation steps.

The Disadvantages

- Limited flexibility for representing future asynchronous or parallel event-processing features.
- May appear verbose when many sequential service interactions exist.
- Less expressive for highly concurrent or real-time distributed behavior.

9.16 Database Specification

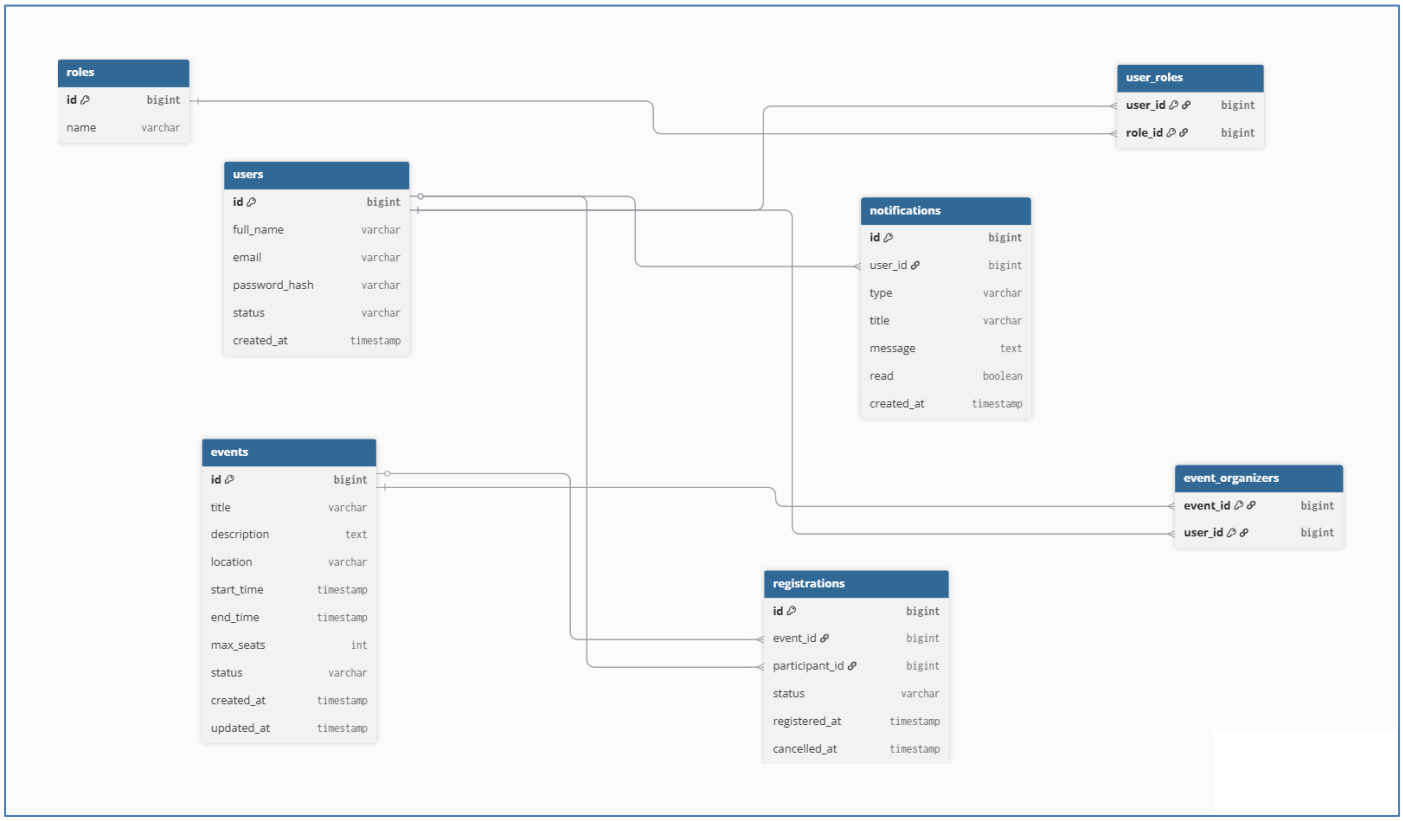
The system uses separated persistence concerns for authentication, events, registrations, and notifications. The following tables/entities summarize the main stored data.

Table / Entity	Purpose	Service Context
users	Stores full name, email, password hash, status, and profile-related values	auth-service
roles	Stores role names such as ADMIN, ORGANIZER, PARTICIPANT	auth-service
user_roles	Maps users to roles	auth-service
events	Stores title, description, location, schedule, max seats, status, organizer fields, timestamps	event-service
registrations	Stores event id, participant id, status, registered_at, cancelled_at	registration-service
notifications	Stores target user, type, title, message, read flag, created_at	notification-service

Database relationships and notes

- A user may have one or more roles through the user_roles mapping.
- An event stores one primary organizer id and a normalized organizer id list representation for multi-organizer support.
- A registration links one participant to one event and preserves status history through status and cancelled timestamp fields.
- A notification belongs to one user and records whether the message has been read.

ERD Diagram


roles
id bigint
name varchar
user_roles
user_id bigint
role_id bigint
users
id bigint
full_name varchar
email varchar
password_hash varchar
status varchar
created_at timestamp
notifications
id bigint
user_id bigint
type varchar
title varchar
message text
read boolean
created_at timestamp
events
id bigint
title varchar
description text
location varchar
start_time timestamp
end_time timestamp
max_seats int
status varchar
created_at timestamp
updated_at timestamp
event_organizers
event_id bigint
user_id bigint
registrations
id bigint
event_id bigint
participant_id bigint
status varchar
registered_at timestamp
cancelled_at timestamp

PART 3: Development Phase (Implementation Details)

User Authentication and Role Management

- Login and Registration: implemented using React forms, Spring Boot REST endpoints, and JWT- based responses.
- Public Registration: the authentication service only allows public self-registration for PARTICIPANT accounts.
- Role Management: ADMIN, ORGANIZER, and PARTICIPANT roles are stored and validated through persistent role mappings and security checks.
- Managed User Creation: admins can create organizer and participant accounts from the admin dashboard via protected endpoints.

Admin Dashboard

- Overview Panel: the admin dashboard acts as the central operational area for event creation and event management tasks.
- User Account Management: admins can create managed accounts and control organizer assignment using dashboard forms and API calls.
- Role-Sensitive Operations: only authorized admin users can perform privileged functions.

Event Management Implementation

- Event creation requires title, description, location, future start time, future end time, capacity, and at least one organizer.
- Event updates are blocked for cancelled events and date validation ensures end time occurs after start time.
- Cancellation updates event status and triggers internal notifications for affected users.
- Rescheduling updates event timing, marks status as RESCHEDULED, and notifies participants and organizers.
- Availability is calculated from max seats and registered participant counts retrieved from the registration service.

Registration Implementation

- Only PARTICIPANT users can create registrations.
- Registration requests are protected by per-event locking to reduce race conditions.
- The system prevents duplicate active registrations for the same participant-event pair.
- Cancellation changes registration status and records cancelled_at.
- Registration and cancellation events trigger notification workflows.

Notification Implementation

- Notifications are stored in a dedicated service and can be created directly or through trusted internal triggers.
- Users can retrieve only their own notifications unless they are admins.
- Unread notifications can be marked as read.
- Internal notification trigger endpoints accept registration-created, registration-cancelled, event- cancelled, and event-rescheduled requests.

Frontend and Data Refresh Logic

- The frontend uses a shared application data context to load events, registrations, notifications, organizer directory data, and managed-user data where appropriate.
- After important actions such as event creation, registration, cancellation, or rescheduling, the frontend refreshes relevant views to keep the interface consistent.
- The My Tickets page displays confirmed registrations only, while the Organizer page filters events based on organizer assignment.

Database and Backend Logic

- PostgreSQL or an equivalent relational database is used for persistent storage in service-specific domains.
- Spring Data JPA repositories provide structured access to event, user, registration, and notification records.
- Feign or client adapters support internal service communication for availability and notification workflows.
- Input validation occurs both at request DTO level and inside service logic for security and data integrity.

PART 4: Complexity & Testing

Are there pairs of Software Quality Factors that are not independent in your system?

Yes.

Integrity & Efficiency

Additional security checks, token validation, and permission enforcement may increase request-processing overhead.

Usability & Maintainability

Highly customized interface behavior may require more frontend logic and testing effort.

Scalability & Simplicity

Introducing more resilient distributed-service patterns can increase architectural complexity.

Calculate the LOC and CCM (Cyclomatic Complexity Metric) for the main functions in your system

Function	LOC	CCM
register() - AuthService	18	3
login() - AuthService	16	3
createEvent() - EventService	24	5
cancelEvent() - EventService	15	3
createRegistration() - RegistrationService	39	7
cancelRegistration() - RegistrationService	18	3
createNotification() - NotificationService	14	3
markAsRead() - NotificationService	14	2

CCM = 1 + number of decision points (if, switch, loops, logical combinations, catch, or branching validation paths).

For the classes in your system, calculate all the following OO Complexity Metrics

- Metrics calculated for EventService and related classes:
- WMC (Weighted Methods per Class): createEvent, getAllEvents, getOrganizerEvents, getEvent, updateEvent, cancelEvent, rescheduleEvent, assignOrganizer, getAvailability plus helper logic produce a relatively high but manageable service weight.
- DIT (Depth of Inheritance Tree): entity and controller inheritance depth is shallow because the project mostly relies on composition and framework base classes.
- NOC (Number of Children): service classes have few or no direct subclasses because the design emphasizes separate components rather than classical inheritance.

- CBO (Coupling Between Objects): EventService couples to EventRepository, RegistrationServiceClient, NotificationServiceClient, AuthUserPrincipal, and DTOs, giving it moderate coupling.
- RFC (Response for Class): service methods invoke repositories, downstream clients, validation helpers, and response mappers, resulting in a moderate-to-high response set.
- LCOM (Lack of Cohesion of Methods): EventService remains reasonably cohesive because most methods focus on event lifecycle behavior.

Considering White-Box Testing, generate a Unit-Testing Test Report for at least 6 main functions in your system.

Function Path	Input Description	Expected Output
createRegistration()	Valid participant, seats available	Registration Success
createRegistration()	Duplicate active registration	Conflict error
createRegistration()	Event full	No seats available error
cancelRegistration()	Active registration	Cancellation Success
createEvent()	Valid event payload	Event created
createEvent()	No organizer selected	Validation error
login()	Valid email and password	Token returned
login()	Invalid credentials	Authentication failure

Considering Black-Box Testing, generate a Functionality System-Testing Test Report for at least 6 main functions in your system

Test Case	Input	Expected Outcome
TC1	Register participant with valid email/password	Pass
TC2	Register participant with duplicate email	Failure with message
TC3	Create event with end time before start time	Failed with validation message
TC4	Participant registers for available event	Pass
TC5	Participant registers for full event	Fail with no seats message
TC6	Admin assigns organizers to event	Pass
TC7	User reads own notifications	Pass
TC8	Organizer requests unrelated event participants	Failing with authorization error

Additional Testing Notes

- Controller tests exist for authentication, events, registration, and notification endpoints in the repository.
- Service tests support verification of business rules such as duplicate registration blocking and event lifecycle behavior.
- Further end-to-end tests can be extended to cover gateway routing, service discovery, and multi- service notification flows.